CodeGuard AI — Presentation Mastery Guide

Everything you need to know to present and defend this project

Task: binary vulnerability detection in C functions (Devign dataset) · **Method:** frozen CodeBERT encoder + trained neural heads (deep transfer learning) · **Hardware:** CPU-only laptop/desktop (AMD Ryzen 5 5600G, 6C/12T, ~16 GB RAM) · **Study:** `cpu_v3`, completed 2026-10-01, test set opened exactly once.

Companion document: *CodeGuardAI_Interview_QA.pdf* (predicted questions + answers). Numbers below are measured on this machine unless marked "published — verify citation".

1. The 30-second summary (memorize this)

> **"I built a deep transfer learning system that flags vulnerable C functions. I take a pre-trained CodeBERT language model, freeze it, and train a small neural head on top of its embeddings — this is transfer learning, not fine-tuning, because the 125M encoder weights never change. I ran three experiments on the official Devign splits, selected the model by validation ROC-AUC, froze the threshold and the weights *before* touching the test set, then evaluated once. My selected model reaches F1 0.647 versus 0.505 for my linear baseline and 0.000 for a majority-class guess, with 93% recall — it catches 1171 of 1255 vulnerable test functions while missing only 84."**

2. Key facts sheet

Item	Value
Project root	`C:\Users\A_R_COMPUTERS\Downloads\CodeGuardAI`
Language / runtime	Python 3.12, local virtualenv `.venv`, PyTorch CPU, Transformers, Gradio
Hardware	AMD Ryzen 5 5600G (6 cores / 12 threads), ~15.3 GB RAM, Windows 11, **no GPU**
Encoder	`microsoft/codebert-base`, revision `3b0952fe…`, 125M parameters, **frozen**
Dataset	CodeXGLUE / Devign (Function-level C vulnerabilities), labels: 0 = non-vulnerable, 1 = vulnerable
Splits (official)	train 21,854 · validation 2,732 · test 2,732
Class balance (test)	1,255 vulnerable (45.94%) vs 1,477 non-vulnerable (54.06%)
Study config	`cpu_v3`: max_length 400 tokens, head [512, 128], dropout 0.2, batch 128, epochs 24, patience 5, lr 0.001, weight decay 0.01, seed 123456
Selected model	C1 MLP head, bundle `final_model/8f0a21195194fc1912b9`, threshold 0.33, ensemble rejected
Encoder speed	3.91 samples/second @400 tokens (measured, `benchmark.json`)
Wall time of full study	~85 minutes (extraction 75 min, heads <1 min each, evaluation + notebooks ~9 min)
Truncation	55.7% of train / 55.5% of validation functions exceed 400 tokens
Test policy	Opened exactly once after freeze; signature stored in `test_opened.json`
Quality gates	24 unit tests passing, `ruff check .` clean
Deliverables	CLI, Gradio web demo, local evidence notebook, self-contained Colab interview notebook, full docs set

3. Problem & dataset

- **Goal:** given one C function, output "vulnerable" or "not vulnerable" (binary classification on the vulnerable class = label 1).
- **Source:** Devign — function-level vulnerability dataset from real C projects (FFmpeg, Qemu), released via CodeXGLUE. Labels come from commit history (vulnerable vs. fixed versions).
- **Official splits used as-is:** 21,854 / 2,732 / 2,732. Splits are never merged or shuffled across.
- **Audit performed** (`codeguard audit`): duplicate and cross-split overlap checks — train∩validation 8, train∩test 4, validation∩test 1 functions in common (out of thousands; reported honestly, never silently removed).
- **Why it's hard:** noisy function-level labels (a function is marked vulnerable even if the bug is elsewhere), long functions (over half exceed 400 tokens → truncated), heavy class ambiguity, and no line-level ground truth.
- **Tokenization:** CodeBERT BPE tokenizer; `max_length = 400` chosen as the accuracy/throughput trade-off for this CPU (benchmark at 128 tokens vs 400 tokens; 400 keeps more context, still fits RAM).

> **Answer to "why 400 tokens?":** it was benchmarked, not guessed. At 400 tokens the encoder runs 3.91 samples/s (full study ≈ 85 min on this CPU), and longer sequences would double runtime for functions where the vulnerability signal is usually local. The truncation rate (55.7%) is reported openly as a limitation.

4. Method — what the model actually is

4.1 Pipeline (one function)

```
source code
   │  CodeBERT tokenizer (max 400 tokens)
   ▼
token ids + attention mask
   │  FROZEN CodeBERT encoder  (125M params, torch.no_grad — weights never updated)
   ▼
last_hidden_state  (400 × 768)
   │  masked mean pooling  (average token vectors, ignoring padding)
   ▼
one 768-d vector
   │  standardization (running mean/std, stored in the checkpoint)
   ▼
 TRAINED HEAD  (LayerNorm, 768 → 512 → 128 → 2, GELU, dropout 0.2)
   │  softmax over 2 classes
   ▼
P(vulnerable)  ──≥ threshold 0.33──►  "Potentially Vulnerable"
```

4.2 Mean pooling formula

$$h = \Sigma_i \, (\text{mask}_i \cdot \text{hidden}_i) \; / \; \Sigma_i \, \text{mask}_i \quad \text{for } i \text{ over the 400 token positions}$$

Padding tokens (mask = 0) are excluded, so the vector represents the actual code content only.

4.3 Why this is transfer learning (and not fine-tuning)

Aspect	Fine-tuning (what we did NOT do)	What we did
Encoder weights	updated by gradients	**frozen forever** — identical before/after training
Trainable parameters	125M+	~0.5M in the head only
Cost	GPU-days	~20–25 seconds per experiment on CPU
Risk	overfits, needs GPU	reproducible, deterministic, hours not weeks

> **Never say "fine-tuned CodeBERT" in the presentation.** Say: "frozen pretrained CodeBERT features + a trained neural classifier head — standard resource-constrained transfer learning."

4.4 The three experiments

ID	Name	Head	Loss	Purpose
C0	`linear`	768 → 2 (linear probe)	cross-entropy	baseline: how far do raw features get us?
C1	`mlp`	768 → 512 → 128 → 2	cross-entropy	main model: non-linear decision boundary
C2	`weighted`	same as C1	class-weighted CE	does penalizing FN-heavy mistakes help?

All three: seed 123456, same features, same batch size 128, early stopping with patience 5 on validation loss (all three stopped early: epochs 9 / 10 / 10 of 24).

5. Experimental protocol (the part that earns marks)

1. **Selection rule (frozen in advance):** rank experiments by **validation ROC-AUC**, tie-break by tuned F1, then recall. Implemented in code (`_selection_key`), not chosen by hand.
2. **Threshold rule (frozen in advance):** grid 0.05 → 0.95 step 0.01 (91 points); maximize F1, tie-break: higher recall → fewer false negatives → closest to 0.50.
3. **Ensemble rule (frozen in advance):** average probabilities of the top two models; keep it only if it beats the single best model's tuned F1 on validation. **Result: rejected** — honest negative result.
4. **Standardization:** head input uses running mean/std computed during training, stored as buffers in the checkpoint (`set_statistics`) so inference never leaks training data stats.
5. **Freeze:** `codeguard finalize` writes the bundle (weights + tokenizer + encoder + specs), records SHA-256 hashes of every checkpoint, then `verify_bundle` re-hashes on every load.
6. **One-time test:** `codeguard evaluate` writes `test_opened.json` ; if evaluation is re-run it just re-displays stored results — no second look, no tuning afterwards.
7. **Smoke vs research:** quick mode uses a separate namespace (`artifacts/cpu_v3/smoke/`) and is never reported as a research result.

Validation results (development evidence)

Experiment	ROC-AUC	Best thr.	Tuned F1	Tuned recall	Tuned accuracy
C0 linear	0.6372	0.22	0.6155	0.9655	0.4758
C1 mlp — selected	**0.6569**	**0.33**	**0.6301**	0.9343	0.5234
C2 weighted	0.6554	0.37	0.6247	0.9242	0.5176

Ranked: mlp 0.6569 > weighted 0.6554 > linear 0.6372 → **mlp selected**. Ensemble of mlp+weighted did not beat mlp alone → **rejected**.

6. Final results (test set, opened once)

Model	Thr.	Accuracy	Precision	Recall	F1	ROC-AUC	TP	FP	FN	TN
Majority guess (always safe)	–	0.5406	–	0.000	0.000	0.500	0	0	1255	1477
Baseline linear probe	0.50	**0.5860**	0.5602	0.4598	0.5050	0.6206	577	453	678	1024
Selected mlp (frozen)	0.33	0.5322	0.4951	**0.9331**	**0.6470**	**0.6560**	1171	1194	**84**	283

What changed vs baseline (selected − baseline)

Metric	Baseline	Selected	Delta
F1 (vulnerable class)	0.5050	0.6470	**+0.1419**
Recall	0.4598	0.9331	**+0.4733**
False negatives (missed vulnerabilities)	678	84	**−594**
False positives	453	1194	+741
Accuracy	0.5860	0.5322	−0.0538
ROC-AUC (threshold-free)	0.6206	0.6560	+0.0354

The one awkward number — own it before they ask.
At the frozen threshold, selected accuracy (53.2%) is *below* the majority guess (54.1%). **Why:** the threshold (0.33) was chosen on validation data only, by the pre-registered rule "maximize F1, then recall" — the right objective when 46% of test functions are vulnerable. That operating point deliberately casts a wide net (recall 93%). A model that flags everything has F1 = 0 and never catches anything by luck; our model has F1 0.647, catches 1171/1255, and its threshold-free ROC-AUC (0.656) beats baseline (0.621) and the trivial 0.5. The threshold was frozen **before** the test set was opened, so no number here was chosen to look good.

Interpretation in one line: the model is a *high-recall screening tool* — it flags more code than it must (precision 0.495) so that almost nothing vulnerable slips through (84 misses vs 678 for the baseline). For a first-pass security triage tool that is the correct trade; the operating point can be moved without retraining.

7. Explainability (token occlusion)

- For a prediction, each token position is replaced with the mask token in batches; the drop in P(vulnerable) measures that token's **signed influence**.
- Influences are summed per source line → top lines ranked by |influence| (default top 10).
- **Positive** influence = removing the token lowered the score (token pushed toward vulnerable); **negative** = it pushed toward safe.
- Explanation targets the **predicted class** (not always class 1).
- Honest limits: it is an **influence explanation, not a guaranteed vulnerability location** — the dataset has no line-level ground truth to verify against. Scores are uncalibrated softmax outputs, not security guarantees.

8. The demo (live UI)

- Run: `run_demo.bat` (or `codeguard demo --open`); binds to `http://127.0.0.1:7860`, auto-falls back to the next free port if busy.
- Loads the frozen bundle, verifies all SHA-256 hashes, prints the URL, opens the browser.
- Features: code editor (C), one-click examples (analysis runs on click), verdict badge with probability bar, metrics summary, line-by-line influence table, machine-readable JSON, Clear button.
- Error handling: empty input, non-string, >200k characters and internal errors all produce a friendly panel — never a raw traceback. Queue is limited so double-clicks are safe.
- Everything runs locally: `127.0.0.1` only, no cloud, no external API.

9. How the code is organized

File	Responsibility
`src/codeguard_ai/config.py`	TOML config, paths, quick/research namespaces, CPU device selection, local HF cache
`src/codeguard_ai/data.py`	official Devign splits, hashing, fingerprinting
`src/codeguard_ai/features.py`	resumable feature extraction, benchmark, occlusion batches
`src/codeguard_ai/models.py`	encoder loading, masked mean pooling, `NeuralHead` (with standardization buffers)
`src/codeguard_ai/losses.py`	cross-entropy + class-weighted variant
`src/codeguard_ai/training.py`	training loop, early stopping, checkpoint save/load
`src/codeguard_ai/metrics.py`	classification metrics, threshold grid search (0.05–0.95)
`src/codeguard_ai/workflows.py`	prepare/compare/finalize/evaluate, selection & ensemble rules, bundle verification
`src/codeguard_ai/inference.py`	`CodeAnalyzer`: single + ensemble paths, occlusion explanation
`src/codeguard_ai/web.py`	Gradio demo (port fallback, formatting helpers, error panels)
`src/codeguard_ai/reporting.py`	EDA plots, audit, local notebook + self-contained Colab interview notebook generator
`src/codeguard_ai/artifacts.py`	JSON I/O, SHA-256 hashing, provenance
`src/codeguard_ai/cli.py`	commands: doctor, prepare, audit, benchmark, extract, train, compare, finalize, evaluate, analyze, demo, notebook, interview
`tests/`	24 unit tests (config, models/standardization, selection, metrics/thresholds, losses, web)
`docs/`	PROJECT_SPEC, RESEARCH_PROTOCOL, ARCHITECTURE, STATUS, DEVELOPMENT_PLAN, AI_HANDOFF, DECISIONS (ADR-001…011), RUNBOOK

End-to-end commands (what was actually run)

```
codeguard prepare      # cache data + encoder (provenance)
codeguard audit        # leakage/duplicate audit + EDA plot
codeguard benchmark    # 3.91 samples/s @400 tokens
codeguard extract      # frozen CodeBERT pass over train + validation (75 min)
codeguard train --experiment linear   # C0
codeguard train --experiment mlp      # C1
codeguard train --experiment weighted # C2
codeguard compare      # validation table + tuned metrics + training plots
codeguard finalize     # selection + threshold + bundle + hashes   (FREEZE)
codeguard evaluate     # one-time test evaluation                  (TEST OPENED)
codeguard notebook     # local evidence notebook
codeguard interview    # self-contained Colab interview deck
codeguard demo --open  # Gradio UI on 127.0.0.1:7860
```

10. Provenance & integrity (say this out loud in the talk)

- Test data never influenced model choice, threshold, ensemble decision, or any hyperparameter — those were all decided on validation data and recorded in `final_config.json` with hashes.
- The evaluation writes `test_opened.json` (signature `5057547dc1ed1815…`); re-running just re-displays stored metrics. Development commands refuse afterwards.
- Every bundle file is SHA-256 verified at load time (`verify_bundle`) — corrupted or swapped weights cannot silently load.
- All reported numbers come from CSV/JSON artifacts on disk, never typed manually.
- Published literature numbers in the deck are labelled **"verify citation"** — you must insert the exact paper references before presenting them.

11. Results compared with published Devign work (context)

Approach	Accuracy	Source
Majority-class guess	0.5406	computed on this test split
BiLSTM	0.5937	published — verify citation
TextCNN	0.6069	published — verify citation
RoBERTa	0.6105	published — verify citation
CodeBERT (full fine-tuning)	0.6208	published — verify citation
Our baseline @0.5	0.5860	this study, measured
Our selected @0.33	0.5322	this study, measured (F1/recall-optimized)

Honest positioning: full fine-tuning of CodeBERT in the literature reaches the low 60s in accuracy; our frozen encoder + trained head sits in the same neighborhood on F1 while running on CPU in hours. We make **no SOTA claim**.

12. Limitations & future work

Limitations (admit first)

- 55.7% of train functions are truncated at 400 tokens.
- Function-level labels are noisy — some "vulnerable" functions have no local bug.
- Encoder frozen → accuracy ceiling below full fine-tuning (~62% in literature).
- Precision 0.495 at the chosen threshold → ~half of flags are false alarms.
- Explanations are influences, not verified vulnerability locations.
- Dataset bias: only FFmpeg/Qemu; may not generalize to other C projects.

Future work (the standard answers)

- GPU → fine-tune CodeBERT end-to-end; hyperparameter sweeps on validation.
- Long-context handling (400-token truncation): sliding windows or longer-context models.
- Probability calibration (temperature scaling) and cost-sensitive thresholds per use case.
- Code-specific encoders, data augmentation, cross-project evaluation.
- Line-level attribution validated against real CVE patches.

13. Presentation script

Slide-by-slide plan (10–12 slides, ~10 minutes)

1. **Title + problem:** vulnerable C code exists; manual review doesn't scale; goal: flag risky functions.
2. **Dataset:** Devign, splits 21854/2732/2732, labels, class balance (46% vulnerable), audit (leak check).
3. **Why transfer learning:** no GPU; frozen CodeBERT + trained head = the honest scope; explicitly "not fine-tuning".
4. **Architecture:** token ids → frozen encoder → masked mean pool (show formula) → standardization → MLP head → softmax.
5. **Protocol:** three experiments; selection by validation AUC; threshold grid; ensemble rule; seed; everything frozen before test.
6. **Validation table:** linear 0.6372 / mlp 0.6569 / weighted 0.6554 → selected mlp; ensemble rejected (honest negative result).
7. **Test results:** big table — F1 0.647 vs 0.505 vs 0.000; recall 93%; FN 84 vs 678.
8. **The accuracy caveat:** address it yourself with the two-table imbalance view + operating-point explanation.
9. **Explainability:** occlusion demo, influence table, honest disclaimer.
10. **Live demo:** paste code → verdict + explanation (or the recorded video).
11. **Limits + future work.**
12. **Integrity close:** **hashes, one-time test, reproducible in ~85 minutes on a CPU.**

Two-minute spoken version

> **"Software vulnerabilities in C are common and manual review is slow, so I built a system that flags vulnerable functions automatically. I used the official Devign dataset with its official splits — about 22 thousand training functions and 2.7 thousand each for validation and test — and audited it for duplicates and leakage first.**
>
> **Because I had no GPU, I used deep transfer learning: a pretrained CodeBERT encoder that stays frozen, with only a small neural head trained on top of its mean-pooled embeddings. That's ~half a million trainable parameters instead of 125 million, which made the whole study reproducible on a CPU in about 85 minutes.**
>
> **I trained three heads — a linear probe, an MLP, and a class-weighted MLP — and selected by validation ROC-AUC using a rule I fixed in advance, together with the threshold grid and an ensemble rule. The MLP won with 0.657 AUC and the ensemble was honestly rejected.**
>
> **Everything — weights, threshold 0.33, hashes — was frozen before I opened the test set exactly once. On test, my model scores F1 0.647 against 0.505 for my baseline and zero for a majority guess, with 93% recall: it catches 1171 of 1255 vulnerable functions and misses only 84. The threshold is recall-oriented, so accuracy at that operating point is 53% — below the 54% majority guess — and I explain that openly rather than hide it: the pre-registered objective was F1, and the threshold-free ROC-AUC is 0.656 versus 0.621 for the baseline.**
>
> **The system also explains each prediction with token occlusion, ships a local web demo, and every metric comes from stored, hashed artifacts — no hand-typed numbers."**

14. Cheat sheet — numbers to memorize

Splits	21,854 / 2,732 / 2,732	**Selected AUC (val)**	0.6569
Test vulnerable / safe	1,255 / 1,477 (45.94% / 54.06%)	**Selected AUC (test)**	0.6560
Threshold (frozen)	0.33	**Baseline AUC (test)**	0.6206
Selected F1	0.6470	**Baseline F1**	0.5050
Selected recall	0.9331 (TP 1171)	**Baseline recall**	0.4598 (TP 577)
Missed vulns (FN)	84 vs 678 baseline	**Δ F1**	+0.1419
Selected accuracy	0.5322	**Majority accuracy**	0.5406
Truncation @400	55.7% train	**Encoder speed**	3.91 /s
Bundle	8f0a21195194fc1912b9	**Full run**	~85 min, CPU only
Trainable params	~0.5M (head only)	**Encoder**	125M frozen

15. Files you may be asked to show

What	Where
Interview notebook (upload to Colab — runs standalone)	`artifacts\cpu_v3\CodeGuardAI_Interview_Presentation.ipynb`
Validation comparison	`artifacts\cpu_v3\results\validation_comparison.csv`
Test comparison + threshold analysis	`artifacts\cpu_v3\results\test_comparison.json`, `threshold_analysis.csv`
Test-opened proof	`artifacts\cpu_v3\results\test_opened.json`
Frozen bundle + hashes	`artifacts\cpu_v3\final_model\8f0a21195194fc1912b9\` + `final_config.json`
Confusion matrices / ROC / threshold plots	`artifacts\cpu_v3\plots\`
Training logs	`artifacts\cpu_v3\train_stdout.log`
Full runbook & decisions	`docs\RUNBOOK.md`, `docs\DECISIONS.md`

16. Glossary (one-line definitions)

Transfer learning	reuse a pretrained model's representations for a new task; here the encoder is frozen and only the head trains.
Fine-tuning	updating the pretrained weights themselves — we do NOT do this.
Linear probe	a single linear layer trained on frozen features; measures feature quality alone.
Mean pooling	averaging token vectors (ignoring padding) into one vector per function.
Precision	of the functions flagged, how many were truly vulnerable (1171/2365 = 0.495).
Recall	of truly vulnerable functions, how many were caught (1171/1255 = 0.933).
F1	harmonic mean of precision and recall; the pre-registered selection objective.
ROC-AUC	probability the model ranks a random vulnerable function above a random safe one; threshold-free.
Threshold	the probability cut-off converting a score into a decision (frozen at 0.33).
Early stopping	stop training when validation loss stops improving (patience 5 epochs).
Truncation	cutting functions longer than 400 tokens (affects 55.7% of train).
Occlusion	masking one token at a time to measure its influence on the prediction.
Class imbalance	54/46 split — why accuracy alone is misleading and F1/recall matter.
Leakage	test information accidentally used during development; prevented by protocol + audit.

Generated for the CodeGuard AI final presentation. All measured numbers trace to artifacts under `artifacts/cpu_v3/`. Literature accuracy figures are reference points requiring exact citations before use.